Research statement

Yitzchak Shmalo. Einstein Institute of Mathematics, The Hebrew University of Jerusalem. November 2026.
Code and write-ups: github.com/yspennstate. Every test, including the ones that went against me: github.com/yspennstate/math-brain-public.

Since the spring of 2026 I have directed AI agents in proving mathematics, in computational problems and on open problems, and in building the memory they work from. I call that memory the brain. It is a graph whose nodes are theorems, lemmas, definitions, proofs, counterexamples, dead routes, recorded mistakes, code and tools, and whose edges say how they relate and how often they were used together. The record of the work is more than a million Claude model calls and more than 17,000 Codex sessions, about half a trillion input tokens and more than 1.5 billion output tokens. The question it raises, how far an agent with a memory can carry a hard problem on a fixed budget, is a question for mathematics as much as for engineering, and the first five sections are about it. The last three are about my other work: random matrices and the stability of deep networks, with Leonid Berlyand; a diffusion limit theorem with Werner Krauth; kernels for emulation; and flows through crypto markets. They are one line of work. The brain is the instrument, the mathematics is what it is for, and the mathematics of the brain itself is a new subject.

1. Agents stop, and they loop

The models are strong for one sitting. What they lack is what a research group has: a memory of what was tried, why it died, what the last person left half done, and the habit of reaching for it at the right moment. Without it an agent proves on Tuesday the lemma it proved on Monday, walks into the dead route the agent next to it just left, and loses the theorem when its context resets. Left alone on a hard problem it loops: the same route, the same partial result, the same next step, for hours. I have watched this in hundreds of logs. So the bottleneck has moved from whether the model can do a step to how many useful steps a system gets from a budget before it stops, repeats itself, or needs me.

I count agency this way: give an agent a task and a budget, measure how far it gets before a person must step in, how often it repeats a step already on record, how often it reaches for a record it needs, and what it pays per right answer. Recall is the plainest part: an agent that finds in one call what another finds in ten searches has nine calls left, and on a long task that compounds. But the record has to be reached for, it has to come back before the agent gives up on it, and something has to turn it into the next step. Most of what I have built is that something.

2. The brain

There are two stores, one inside the other. The **math brain** is public and typed. Its cards are definitions, theorems, lemmas, propositions, corollaries, proof techniques, worked proofs, counterexamples, verified computations, open problems with their frontier, and dead routes with the step at which they failed; each carries a kind, a domain, its sources and a status (verified, provisional, superseded). Its links are the relations a mathematician uses: *depends on*, *proves*, *refutes*, *generalizes*, *specializes*, *analogous to*, *bridges to*, *cites*, *defines*, *supersedes*. A wrong card is never deleted; a revision supersedes it, so the next agent sees the error and the fix.

Of the 1,137 superseding cards, 139 correct what a card said for a stated reason. The private store held 90,596 cards on 28 October 2026 with 76,434 edges derived from the cards' own citations and definitions; the public snapshot of 1 November has 13,344 cards and 23,518 typed links.

The **research brain** is the larger private store the math brain sits inside: about 41,000 written memories (results, rules, mistakes with their mechanism, references), the cards, prompts, tools, files, keywords, and 367 compiled reflexes. Its edges are of two kinds. Typed links written by agents, as above. And learned edges that nobody writes: when two nodes are used together in one action, or one right after the other, the edge between them strengthens, and it fades when they stop being used. On 1 November 2026 the main graph held 349,065 nodes and 15.5 million learned edges, built from 15.3 million recorded events across 5,747 sessions; the whole system holds about 415,000 nodes. It connects probability, deep learning, random matrix theory, finance, Bitcoin, the agents' own workflows and the building of the brain itself, by edges that in many cases nobody knew to draw. When a theorem and a piece of code become linked because agents kept using them together, the brain is teaching agents what is connected.

I call the brain a mathematical achievement in two senses. The content: thousands of agent sessions, directed by me, read the literature of probability and deep learning, wrote the cards, proved and refuted what they could, and connected results across fields by edges that were in no book. And the object: a weighted, typed, temporal graph with a dynamics on it, built by a population of agents, that can be studied with random walks, spectral theory and network science, and that I study that way. Nothing in the method is specific to mathematics.

3. The mathematics of the brain

The store. A *card* is one unit of record: a type (definition, theorem, lemma, proof, counterexample, verified computation, dead route, open problem, recorded mistake) and a text. A *link* is a typed, directed edge from one card to another (depends on, proves, refutes, generalizes, cites, bridges to), with a positive weight. The store is the weighted directed graph $G = (V, E, w)$: V the cards, E the links, $w(i, j)$ the weight of the link from card i to card j. Its adjacency matrix A has $A_{ij} = w(i, j)$, zero where there is no link; D is the diagonal matrix of row sums, $D_{ii} = \Sigma_j A_{ij}$, the total weight leaving card i; and $P = D^{-1}A$ is the row-normalized matrix, whose row i says what fraction of card i's weight goes to each neighbour, so every row sums to one. A card with three links of weights 2, 1 and 1 has P entries 1/2, 1/4, 1/4 on that row.

Recall is a walk. A question q is a string. Each card gets two seed scores. The lexical score $\ell_i(q)$ is BM25 over the card's text, the standard term-weighting score: a word of the question counts for more the rarer it is across the store and the more often it occurs in the card, with the card's length normalized out. The semantic score $m_i(q)$ is the cosine between an embedding vector of the question and the stored embedding vector of the card, so two texts that say the same thing in different words score high. Each score is standardized over the candidate cards (its mean subtracted, divided by its standard deviation, so the two are on one scale), and the seed vector is

$$s_i(q) = z(\ell_i(q)) + 0.3\, z(m_i(q)),$$

the weight 0.3 chosen on a held-out bank. Activation then spreads along the links, two hops at most, and cards are ranked by

$$a(q) = s(q) + \beta_1 P^{\mathrm{T}} s(q) + \beta_2 (P^{\mathrm{T}})^2 s(q).$$

The entry $(P^{\mathrm{T}}s)_i = \Sigma_j P_{ji}\, s_j$ is what card i collects from the cards that link to it: each linking card j passes on the fraction of its weight that points at i, times its own seed score. A card the question's words never touch, linked from two well-matched cards that each give it half their weight, receives $\beta_1(s_j + s_k)/2$ and can enter the candidate pool that way. The normalization matters. Under A itself a card passes its whole score to every neighbour, so a closed clique of sixteen junk cards, 1.4 percent of a store with 92 percent of its edges internal, fed itself on every question and climbed; under P each card distributes one unit of weight however many links it has, and the clique went quiet. Over 7,440 trials with real prompts a new memory reached the top ten by spreading alone zero times; on 584 held-out questions spreading moved first-eight hits from 298 to 307 and found no card the words had missed. So the walk buys membership of the candidate pool and the seed scores decide the order; production runs the links as a re-ranker only (section 5).

Use has a weight. The second graph is the use graph: its nodes are the memories and cards, and two nodes are joined when they were used in the same action. A node's weight is its base-level activation in the sense of ACT-R and an edge's weight is a decaying count of co-uses:

$$B_i(t) = \ln \Sigma_k\, (t - t_{ik})^{-d}, \quad w_{ij}(t) = \Sigma_k\, e^{-(t - t_{ijk})/\tau},$$

where $t_{i1}, t_{i2}, \ldots$ are the times node i was used before t, t_{ijk} the times i and j were used together, d the decay exponent (ACT-R's 0.5) and τ a time scale. Both grow with use and fade with age. With $d = 1/2$ and time in days, one use yesterday contributes 1 and ten uses a year ago contribute $10/\sqrt{365} = 0.52$, so the memory touched yesterday outranks the one that was popular last year. The graph this builds is not random: on 28,288 nodes the clustering coefficient is 0.255 against 0.020 for a rewired graph with the same degrees, and the mean path length 5.28 against 3.93, a small world with ratio about nine.

Silence is a rule. An agent cannot search for what it does not know it needs, so a layer of hooks reads every prompt and action before it runs and puts the nearest cards, the mistakes recorded in that territory and the reflexes compiled from them in front of the agent, in 0.20 seconds on a prompt. A *reflex* is a scar made executable: a trigger pattern and a rule, compiled from a recorded mistake. The layer speaks only when it should. Let $a_{(1)} \geq a_{(2)}$ be the two largest activation scores for the prompt. The rule is

$$\text{serve iff } a_{(1)} \geq \theta \text{ and } a_{(1)} - a_{(2)} \geq \gamma\text{:}$$

something matches well, and one thing matches clearly better than the rest. A prompt about an engine's closing rule matches one recorded mistake far above everything else and that note is served; a prompt that says only "the test" matches twenty memories about equally and the layer stays silent. On 2,000 questions with an answer in the store and 60 without, the rule's score separates the two with AUC 0.997 (the chance that a random answerable question scores above a random unanswerable one) and 6.4 percent of the answerable ones were wrongly met with silence. At most two reflexes fire per action, with back-off, and a nightly judge scores each fire so the ranking learns. Every served note carries a line saying how far to trust it, because a note says "this resembles a place we have been", not "this is right": served notes helped on one class of

faults and hurt on another, where a note about a cited result made a wrong use of it look legitimate.

Growth has a gate. Fix a bank of held-out questions with known answer cards and let acc(S) be the fraction whose answer card ranks in the first eight when recall runs over the store S. A packet Π of new cards and links lands only if it costs nothing on the bank: write Π iff acc($S \cup \Pi$) $\geq$ acc(S), measured on a copy. A new card is reachable by the next call after it lands. A nightly pass consolidates the day into memories, judges the reflex fires, grows the use graph from what was used together, and attaches orphan cards by similarity; 23 percent of memories once had no edge at all, invisible to the walk.

The frontier layer, against loops. Each staffed open problem has a dossier: the statement, the frontier papers and where their proofs fail, the obstruction, what is settled, which routes are dead and why, and named targets in order. A *route* is a proof strategy written as a sequence of steps; a *dead route* is one with a recorded reason it cannot close. A worker starts from the failure point, checks its route against the settled facts and the dead routes, and is stopped on a match unless it writes a concrete reason its route differs. Only two kinds of claim count as progress: a step of the frontier proof goes through where it failed, or the statement is proved on a new part of its range. A literature answer outranks everything. On 28 October the layer covered 64 staffed problems with 2,623 routes and 2,197 claims, from a board of about 400 open problems, half in probability and half in the theory of deep learning.

Many agents on one record. Twenty to forty agents work on this record at once, in two model families, for weeks, and the costliest failures are not missed memories: two agents editing one file, one re-deriving what another finished an hour ago, an order of mine reaching one terminal and not the others. The habits a research group has are written as protocols enforced by code. The shared things (a file, a folder, a repository, a topic) form a set R ordered by containment, a folder above every file in it. A *lease* is a triple (r, m, t): a resource, a mode $m \in$ {read, write, exclusive} and an expiry time. Two leases conflict iff their resources are comparable in the order (one contains the other, or they are equal) and not both are reads; so a write lease on a folder blocks a write on any file inside it and lets reads through. A handoff is two-phase, offer then accept. Every ask is registered on one checklist with the agent that took it. Each agent keeps a one-line note of what is done and what comes next, so a crash costs minutes, and before reporting derives each number that matters a second way. The hooks that carry these habits are built by the agents as the brain grows: the brain improves the agents, and the agents improve the brain.

Speed is part of it. A memory an agent cannot reach quickly is a memory it stops calling. The programme that took a recall from 26.8 to 7.4 seconds and the prompt hook from 1.24 to 0.20 seconds is engineering, but the reason for it is in section 4: under load the accuracy of a recall holds while its wall time goes from 8 seconds to 9 minutes at 6 seconds of CPU, and the agents stop reaching for it.

4. The tests

The arms. Every test compares arms that differ in one thing. *Bare*: the model alone, no tools, no files. *Repositories* (the git-only arm): the same model with a shell and the checked-out repositories, every file the memory was built from, free to grep, read and run anything, and no recall tool. *Memory*: the repositories arm plus one tool, recall over the store; in the tests on git

repositories the memory arm is allowed exactly one recall call per question and then works as the repositories arm does. The same questions, the same model, the same time cap in every arm. A *pair* is one question answered in two arms, and a pair is *discordant* when exactly one arm is right; the p values are exact binomial tests on the discordant pairs (the chance of a split at least that lopsided if the two arms were equally good). Every test is in the public repository with its design, aggregates and raw rows.

The strongest. 988 questions on the research record, 23 and 24 October 2026, Claude agents, 600 seconds an item, all three arms on the same questions. Bare, 18.9 percent. Repositories, 82.0 percent. Memory, 95.4 percent. On the discordant pairs the memory arm beat the repositories arm 146 to 13 ($p = 1.2 \times 10^{-29}$) and the bare arm 757 to 1. The memory arm answered in about half the time (120 seconds an item against 211 for the repositories arm) and ran out of time a fifth as often (3.7 percent against 17.5). The mechanism is visible in the pairs: on the 806 items that both the memory and the repositories arm finished, accuracy was tied, 99.3 and 99.4 percent, so the lead is time; that set is selected by finishing, so it says where the lead sits, not why. A memory does not make the model smarter on a question it can finish; it makes the question finishable. It is also cheaper where it wins: a memory session read 20,000 tokens against 80,000 for a repositories session, $0.053 against $0.092 a session, $0.074 a right answer against $0.153. The 77 proof audits whose answers are stored nowhere sat at the ceiling in every arm, about 98 percent: the memory helps on what we recorded and is neutral on reasoning that is not about our records. With Codex agents on 394 of the same items the repositories arm led the memory arm, 97.0 against 94.7 ($p = 0.14$); 62 of the 69 Codex memory-arm misses were sessions in which the recall tool never loaded, because Codex drops a tool server that is not ready when the turn starts. Codex rows that did call recall were right 96.8 percent of the time.

The rest. My first test, twenty items, memory against bare: 19 of 20 with the memory, 9 of 20 without ($p = 0.002$); every bare miss was a fact that lives only in our store. Two thousand new questions on files in 113 git repositories, Codex agents, memory against repositories: 93.7 against 91.4 percent (83 to 43, $p = 4.6 \times 10^{-4}$, $n = 1{,}726$), and on an earlier bank 90.5 against 86.1 (78 to 36, $p = 1.0 \times 10^{-4}$, $n = 967$). A thousand items whose answer must be recalled, memory against repositories: 1,000 against 538. One hundred and sixty proof audits with planted faults, the right served notes against scrambled ones: 128 against 117 ($p = 0.043$), replicated, the only test so far of whether served notes make a checker better rather than faster. Ranking without agents on 584 held-out questions, the walk against the words alone: 317 against 298 (27 to 8, $p = 0.002$). Depth bought nothing: after 76,000 new links, spreading one step out scored the same as not spreading at 4.5 times the time, so the default is depth 0 and the links are for the reader. And the test the memory lost: a round of the 2,000-question test on a machine with a load near 340, one recall taking 700 seconds or more. The memory arm all but stopped calling it and did worse than the repositories arm, 39.9 against 46.5 percent. That failure is why speed is in section 3.

5. Results, and the theory I want

Agents working from the store, under the frontier layer, have settled open problems in probability: finite-state counterexamples to Ross's second queueing conjecture (arXiv:2609.25084), Vu's Conjecture 9.5 (false), Lyons and Peres Question 16.7 (false), Aldous

and Fill Open Problem 7.30, Kosygina and Zerner Problem 5.3, the outstanding numerical conditions in the storage-capacity theorem for the Ising perceptron (arXiv:2607.11010, interval arithmetic, partly checked in Lean 4), and about 45 further repositories of the same kind, each paper with a declaration of what the systems did and audited by independent agent runs. The system itself is written up as a full paper (*Math Brain: shared, auditable memory for multi-agent mathematical research*, October 2026). The store corrected itself 139 times with a stated reason. And the link graph has a spectrum: on 15,087 links among 6,532 cards, 39 of 39 leading eigenvalues of the normalized link matrix clear a rewired null with the same degrees, the outliers are local (a median 76 percent of their mass on twenty cards), and linked cards share about ten times as many rare words as random pairs. The links carry structure the words already see, which is why they only re-rank.

The theory I want is a capacity theorem. Take a store of N cards whose words and links are generated from K planted topics with a correlation parameter ρ between the two channels. Conjecture: there is a threshold ρ^* below which the walk over the links strictly improves held-out recall over the word channel, and above which it only re-ranks; my stores sit above it. The measurement is to break the correlation on purpose in a held-out copy and watch the gain appear. The second theorem is about loops: model a worker as a Markov chain on routes with the dead routes absorbing, and show how the expected number of useful steps per budget scales with the fraction of the frontier on record. The counts in section 3 are the data for it; the paired test is the next year's work.

6. Random matrices, stability and the training of networks

My thesis with Leonid Berlyand used random matrix theory to say what a trained network keeps. Write a trained layer, an $N \times M$ weight matrix, as $W = S + R$: S of low rank, the signal, and R noise with independent entries of variance σ^2/N. The singular values of a matrix are the square roots of the eigenvalues of $W^{\mathrm{T}}W$, the factors by which it stretches space along its principal directions. For R alone the Marchenko-Pastur law says that as N grows with $c = M/N$ fixed, the histogram of singular values settles on a fixed density supported on $[\sigma(1 - \sqrt{c}), \sigma(1 + \sqrt{c})]$: for a square matrix with $\sigma = 1$ the bulk runs from 0 to 2, and a singular value at 3 is not noise. Fitting the bulk means estimating σ from the trained layer so that this density matches the histogram of its singular values; the singular values inside the bulk are then read as noise and the ones that separate from it carry S.

> **Theorem** (Berlyand, Sandier, Shmalo, Zhang, *J. Mach. Learn.* 2024). Fit the bulk to each trained layer and remove the singular values inside it. The pruned network's loss stays within an explicit bound of the trained loss, depending on the removed singular values and the Lipschitz constants of the activations; the cut cannot hurt accuracy by more than that bound, and in practice it often helps.

With Berlyand, Bourdais and Owhadi the same cut, with a regularizer that pushes bulk and spikes apart during training, was carried to vision transformers and modern CNNs with public code (arXiv:2503.01922, 2606.02608). When the signal rank grows with N the question becomes mesoscopic, which is where my recent preprints sit: outlier singular values and singular subspaces of rectangular spikes under a subspace local law (arXiv:2608.19594) and the additive case.

The stability work with Berlyand runs the other way: not what a network keeps but whether its map is stable at all. A deep Gaussian ReLU network is the iteration $x_{t+1} = \mathrm{ReLU}(W_t x_t)$, $\mathrm{ReLU}(u) = \max(u, 0)$ coordinate by coordinate, with independent Gaussian matrices W_t. Such a network is piecewise affine: on each region of inputs where the pattern of active units (which coordinates are positive at each layer) stays fixed, the map is one affine map, and I call that pattern an *affine orthant*. The natural tool is the gauge orbit $W \to \Sigma W \Sigma$ over diagonal sign matrices Σ, which leaves the network's law invariant and makes the average over signs exact.

Theorem (global stability, finite width). For L independent Gaussian layers of width n the number of affine orthants the network can realize is at most $\Sigma_{j \le n}\, C(nL, j) \le e^{nL\, H(1/L)}$, and the stability margin K, the quantity whose staying above one is what the proof needs for the network's map to contract, drops to one or below with probability at most the Chernoff bound $P(K \le 1) \le L \exp(-n\, I(1/\alpha))$, with $I(c) = \sup_s[-sc - \log((1 + (1 + 2s)^{-1/2})/2)]$. Here H is the binary entropy, so the orthant count is exponentially smaller than the 2^{nL} sign patterns a priori, and I is a rate function; at $\alpha = 3$, $L = 10$, $n = 1000$ the bound is 8.7×10^{-6} where the second-moment (Chebyshev) bound gives 0.45. With biases of variance σ^2 the layer variance obeys $q_t \to \sigma^2 + (1 - \sigma^2)\, 2^{-t}$.

Theorem (one matrix, many steps). Let one Gaussian W with entries of variance $1/N$ be applied T times with ReLU from a unit start. For every fixed t the coordinates of $\sqrt{N}\, x_t$ have the empirical law of $N(0, 2^{-(t-1)})$, $||x_t||^2 \to 2^{-t}$, half the units are alive, and two trajectories driven by the same W become parallel, with angle $\sim 3\pi/t$. The proof is an innovation representation of the reused matrix (a Gram-Schmidt driving sequence, a uniform law of large numbers, an operator-norm bound).

The reused-matrix result is a rigorous finite-width, multi-step version of the ordered phase of the edge-of-chaos analysis, where $\chi_1 = \sigma_w^2\, E[\varphi'(\sqrt{q^*}\, Z)^2]$ (q^* the fixed-point variance of a unit, Z standard Gaussian, φ the activation) is the factor by which a small perturbation grows per layer: below one it dies, above one it is amplified. In the ordered phase $\chi_1 < 1$ I also have the exact constant in the smallest eigenvalue of the neural tangent kernel of a deep tanh network, $\lambda_{\min} \sim \kappa\, L\, \chi_1^{\,L}$, with the strict positivity that the order-of-magnitude result of Xiao, Pennington and Schoenholz left open (2026). With Berlyand, Sarapin, Slavin and Sodin (arXiv:2608.14638) we studied the same question for randomly initialized autoencoders as a dynamical system: the fixed points of the map and the edge of chaos where the Jacobian's spectral radius crosses one. The open question I am working on with Berlyand is the global Lipschitz constant of the tied-weight network. Three more results sit on the same line. The uniform doubling condition (*Ann. Math. Artif. Intell.* 2024): if the data measure satisfies $\mu(B(x, 2r)) \le C\, \mu(B(x, r))$ uniformly, a classifier with cross-entropy below a level has accuracy above a level explicit in C, so accuracy cannot collapse while the loss keeps falling, and without the condition it can. The law of robustness (arXiv:2607.07778): any two-layer interpolant of n points in dimension d with p parameters has Lipschitz constant of order $\sqrt{(nd/p)}$, the Bubeck-Sellke bound now with arbitrary weights. The extreme least singular value (*Bull. Aust. Math. Soc.* 2026): a lower bound on the least singular value holding simultaneously over every m-row submatrix of an $N \times n$ Gaussian matrix, the entropy of the choice paid in the exponent, which makes phase retrieval

stable under adversarial deletion of measurements. And the Ising perceptron (arXiv:2607.11010) closed the numerical conditions in the storage-capacity theorem with interval arithmetic, partly in Lean 4.

With Werner Krauth (ENS Paris; the preprint has not yet been released). A diffusion model samples a target law by running a noising process forward and a reverse process back. Krauth and coauthors built a backward sampler that needs no learned score: the target is known only through p sample points, the *patterns*, and the backward path is at each time a mixture of Gaussian bridges (Brownian paths pinned to end at a pattern), each weighted by the posterior probability of its pattern given the current state. Run it with p patterns and, driven by the same Brownian noise, with the true target in place of the patterns ($p = \infty$); the question is how far apart the two paths are. *Stable* convergence below means convergence in law jointly with the driving noise, which is what allows the limit's own randomness to be independent of that noise.

> **Theorem.** For sub-Gaussian targets and every strip $[\tau_0, \beta]$ away from the endpoint, $\sqrt{p}\,(X^{(p)} - X^{(\infty)})$ converges in C^1, stably with respect to the noise, to $\Xi_\tau = \int_\tau^\beta \Phi(\tau, r)\, G(X^{(\infty)}_r, r) \,/\, (r\, b(X^{(\infty)}_r, r))\, \mathrm{d}r$, with Φ the propagator of the linearized drift and G an isonormal Gaussian field over L^2 of the target, independent of the noise. For a Gaussian target a change of time and scale removes every parameter: the fluctuation is a universal process Z with Hermite expansion $Z_T = \Sigma_{n\geq 1}\, A_n(u_T)\, \xi_n$, $\mathrm{E}\, A_n(U)^2 = 2U^{n+1}/(n+1)$. At the endpoint the variance diverges logarithmically yet Z_T converges almost surely, with tail $\mathrm{P}(|Z_\infty| > x) \sim 1/(x^2 \log x)$, a law of the iterated logarithm with constant $8 - 4\sqrt{2}$, and Hölder continuity below exponent $1/4$ and not at it.

For a non-Gaussian target the fluctuation solves a bridge equation $\mathrm{d}Z = (\tilde{m}(Z, u) - Z)/(1 - u)\, \mathrm{d}u + \mathrm{d}B$, with $\tilde{m}$ the posterior mean of the pattern, so the whole object is a kernel smoother run along a diffusion. Every claim went through an exact replay of the reference scripts (agreement to 2×10^{-15}), a finite-p simulation against the limit law, and machine-checked algebra; the second check caught a dropped constant within the hour.

Kernels. At NASA's Jet Propulsion Laboratory in 2024 I worked on neural emulators of radiative transfer for imaging spectroscopy (EMIT, AVIRIS, OCO-2) and on low-rank and sparse kernel methods, and the kernel work continued. The estimator is a neural mean with a kernel correction. Given training inputs $X = (x_1, \ldots, x_n)$ with outputs y, a trained network g_θ and a kernel k, with $k(X, X)$ the $n \times n$ matrix of kernel values between training points and $k(x, X)$ the row of values between a new point and them, the prediction is $f(x) = g_\theta(x) + k(x, X)(k(X, X) + \lambda I)^{-1}(y - g_\theta(X))$: the network carries the mean and a Gaussian-process regression on the residuals $y - g_\theta(X)$, with ridge λ, carries what the network misses; the kernel matrix is made low rank so the $n \times n$ solve scales. The metric $k(x, x') = \exp(-|M(x - x')|^2)$ can be learned (kernel flows, a low-rank Mahalanobis M); the finding of arXiv:2609.00389 (2026, on public data) is that learned metrics reproduce a hand-tuned one and do not beat it: representation and data are the order-of-magnitude levers, the metric a constant-factor one. Better kernel-with-network methods, pruning and robustness are exactly what the agents now work on from the memory, with the random-matrix theorems, the kernel results and the training recipes as linked cards.

7. Flows through crypto markets

A smaller, newer line, and the same mathematics. Laundered money moves through tens of hops across billions of addresses, and the question is a flow question on a graph too large to hold: which mass at a sink came from a given source, through every path of length up to sixty, with mixing and change outputs accounted for. I have a one-sweep tracer for all-hop flows on the Bitcoin transaction graph, and the plan is the conservation law written out as a linear system on the transaction DAG, solved by a pass in topological order with the same degree normalization that recall uses, so that a hub does not swallow the flow. The agents will carry it the way they carry the open problems: from the store, under the frontier layer, with every dead route filed. The application is to criminal flows, and the mathematics is the mathematics of section 3.

8. What comes next

The next years of this work are the brain, in three concrete senses: making recall faster and more exact, making the agents that run on it more autonomous, and taking the mathematics of flows from the Bitcoin work into the memory and back. Each carries a measurement, because that is the standard section 4 set.

Recall, ten times faster and more exact. The brain is the slowest row of its own scoreboard: a cold recall takes seconds on a laptop and the math store answers in a fraction of that, and what is slow is not the walk but the loading, the vector table read whole, an index rebuilt when it is a few minutes old, a process started afresh for every call. The engineering fixes are under way and measured (a lazy load of the vector rows cut the load three times, a resident process buys a factor of two), but the order of magnitude is in the mathematics. Three pieces. First, the spreading step is a product $P^{\mathrm{T}}s$ over the whole store when only a few hundred rows matter; bounding the walk to the cards whose activation can still clear the serve threshold θ turns it into a frontier expansion whose cost is the size of the active set, not N. Second, the two seed scores are standardized against the whole bank on every call; a running estimate of their moments makes that step constant in N. Third, the weight 0.3 between the lexical and the semantic score and the decay d in the base-level activation are set by hand; both can be fitted by maximum likelihood on the judged bank (which card was the answer, which cards were served) and held out, and the held-out accuracy is the test. The bar is written down: a tenth of the median latency on the same bank with accuracy held within a point, before and after on the same questions.

Agentic workflows, and the frontier used properly. The frontier layer records routes and dead routes, but an agent still chooses its next route by reading; the choice should be a decision rule with a measured regret. Model it as a bandit over routes: each route r has an unknown probability p_r of yielding a useful step, the dead-route record is the observations so far, and Thompson sampling over the Beta posteriors gives the order in which agents try routes. The record is then the posterior, shared across agents and across days, and a route is dead when its posterior mass above a floor falls under a set value, not when one agent gives up. The paired test the frontier lacks is one experiment: the same open problems, agents with the record against agents without it, useful steps per budget as the statistic, the exact binomial as in section 4. The second half is the working loop itself. An agent recalls, acts and records; the hooks that fire reflexes, judge their precision overnight and silence the ones judged wrong are the first learned part of that loop. I want the whole loop learned: which memories to serve at which step, when to

hand off, when to stop, fitted on the record of what agents did and what came of it, with the serve rule's AUC as the gate so nothing lands that fires worse than the rule it replaces. The tests scale with it: the 988-item bank is one group's record, and a second group's record, with its own agents and no shared history, is the test of whether the gains are the mechanism or the group.

Flows from Bitcoin to memory, and back. The two graphs differ in every particular, a transaction graph with value conserved at each node against a card graph with activation spread by use, and agree in the mathematics: a conserved or normalized quantity pushed along directed edges on a graph too large to hold, where the question is what reaches a given node through paths of bounded length. The Bitcoin tracer taught me three things the memory does not yet do. The flow is conserved: at each transaction the inputs' taint is split over the outputs by a rule (proportional, first-in-first-out, or poison), and conservation is what makes the whole computation one pass in topological order. Recall's spreading loses mass at every step and has no conservation law, so a card's activation after k steps is not comparable with another's after j, which is one reason the walk only re-ranks. A conserved variant, a unit of activation split over the links by weight and summed at each card, gives a quantity with a meaning (the share of the question's mass that reaches the card) and a stopping rule (stop when the unsent mass is below ε), and the capacity conjecture of section 5 becomes a statement about where that mass goes. The second is the frontier: a tracer over billions of addresses keeps only the active set and expands it hop by hop under a budget, and that bounded expansion is the faster recall above. The third is incremental indexing: the chain grows at the front and the index is updated for new blocks only; the store grows the same way, and a new card should enter the walk by updating the rows it touches, not by a rebuild.

In the other direction the memory has three things the tracer wants. Use has a weight: the base-level activation $B_i(t)$ says that a path travelled recently and often is the path to look at first, and on the transaction graph the same decay over the times a path carried flow ranks the hops to expand when the budget is short; recency is exactly what an investigation wants. Silence is a rule: the tracer should attribute a sink to a source when the leading source's share clears a threshold and leads the second by a margin, the two-score rule of section 3, and the same AUC measures it against labelled cases. And dead routes: a region already traced and found to carry nothing to the sink is filed and not traced again by the next agent, and across a fleet of agents on one chain that is where the time goes. The object I want is one library of flow primitives, bounded-frontier propagation, conservation with a splitting rule, use-weighted expansion order, a two-score stopping rule and a shared record of dead regions, with the memory and the chain as its two instances and a test bank for each.

The mathematics that goes with it. The theorems of section 5 are the frame: the capacity threshold ρ^*, and the Markov chain on routes with the dead routes absorbing, whose expected useful steps per budget as a function of the fraction of the frontier on record is what the paired test estimates. Two more questions come from the flow view. Mixing: the walk P on the use graph is a Markov chain, its spectral gap says how many steps carry information before the walk forgets the question, and the depth at which recall stops improving (depth 1 on my stores) should be readable from that gap; the small-world numbers of section 3 are the first data. Conservation on a graph with cycles: the one-pass tracer is exact because the transaction graph is acyclic, the use graph is not, and the conserved variant of recall needs the absorbing or the damped version, with the damping chosen where the series converges and the mass still has its

meaning. Both questions sit on random-graph and random-matrix ground, and I expect the spectral results to be where the two halves of this statement meet.

How the work is done. All of it runs the way section 4 runs: a bank, two arms, a paired exact test, before and after on the same questions; agents on the record under the frontier layer, with every dead route filed; the public repository carrying the tests and the measurements as they land. The random-matrix and stability work with Berlyand, the Krauth manuscript and the kernel work continue beside it, and the agents now work those problems from the same memory, so the brain is both the subject of this plan and the instrument the rest of it is done with.

9. Why this, and what could go wrong

I am a probabilist who built an instrument because the problems needed it, and the instrument turned out to be a mathematical object. The risk is plain and I state it: the frontier layer and the working loop have counts and no paired test yet; the capacity theorem is a conjecture with a measurement plan; most of the 2026 papers are not yet refereed. What I can say is measured: on the questions a research group asks of its own record, the memory arm is right 95 percent of the time where the repositories arm is right 82 and the bare arm 19, in half the time, at half the cost, and the one time it lost, I know why. The next year is the two theorems, the paired tests of the frontier layer, and the mathematics the agents are now doing from the store.